RNN과 CNN

시퀀스 데이터와 이미지 데이터의 처리

이종혁(경희대학교 미디어학과)

참고: wikidocs.net/book/2155

왜 RNN이 필요한가? — 시퀀스 데이터의 특수성

일반 신경망(FFNN)은 입력 간 순서를 고려하지 않음. 하지만 언어, 음성, 주가 같은 시퀀스 데이터는 순서가 핵심 정보.
"나는 학교에 간다"와 "간다 학교에 나는"은 같은 단어지만 의미가 다름. → 순서를 기억하는 신경망이 필요.

1 시퀀스 데이터란?

순서가 의미를 가지는 데이터

텍스트: 단어의 나열 순서가 의미 결정
음성: 시간 순서로 파형이 변화
주가: 과거 가격이 미래에 영향
날씨: 어제 기온이 오늘에 영향

2 FFNN의 한계

모든 입력을 독립적으로 처리
→ "이전에 뭐가 나왔는지" 기억 못 함

입력 길이가 고정되어야 함
→ 문장마다 단어 수가 다른 텍스트 처리 불가

RNN: 이전 단계의 출력을 다음 단계
입력으로 전달 → "기억"을 가진 신경망

RNN (Recurrent Neural Network) — 기본 구조

RNN의 핵심 아이디어

은닉층의 출력이 다시 자기 자신의 입력으로 순환(recurrent)

각 시점 t에서:

h_t = tanh(W_hh * h_(t-1) + W_xh * x_t + b)

h_t: 현재 은닉 상태 (현재의 "기억")
h_(t-1): 이전 은닉 상태 (과거의 "기억")
x_t: 현재 입력
W_hh, W_xh: 학습되는 가중치

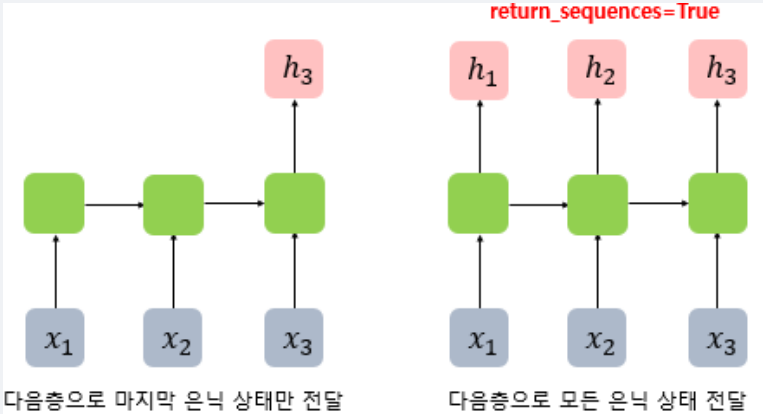
비유:
책을 읽을 때 앞 내용을 기억하면서
현재 문장을 이해하는 것과 같음

RNN 활용 분야

텍스트 분류 / 감정 분석
기계 번역 (Seq2Seq)
음성 인식
시계열 예측 (주가, 날씨)
텍스트 생성

RNN의 한계

장기 의존성 문제
시퀀스가 길어질수록 초기 정보가 후반부에 미치는 영향이 사라짐
기울기 소실 문제
역전파 시 그래디언트가 시간에 따라 기하급수적 감소



$$h_t = tanh(W_{hh}h_{t-1} + W_{xh}X_t + b_h)$$

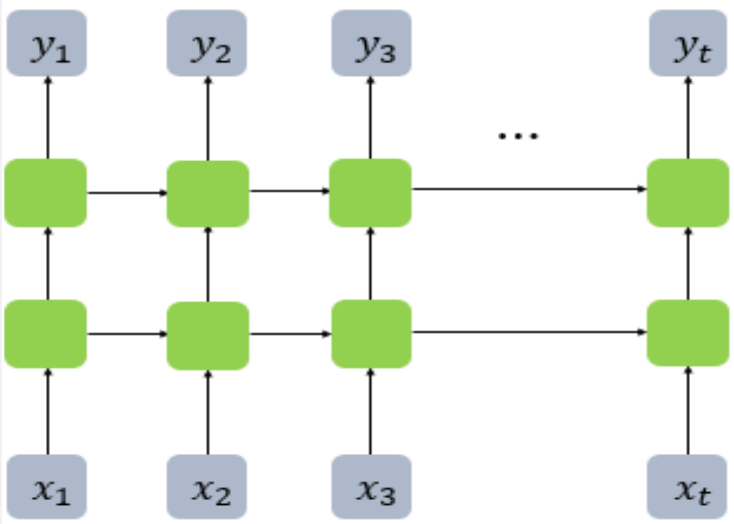
Deep RNN & BiRNN

Deep RNN (**Deep Recurrent Neural Network**) : 기본 RNN을 수직으로 쌓아 더 깊은 표현력을 갖춤

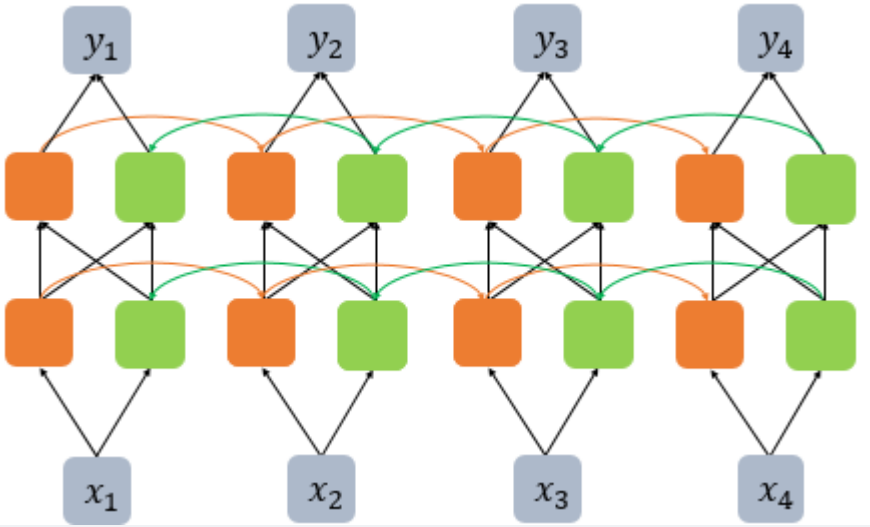
BiRNN (**Bidirectional Recurrent Neural Network**): 시퀀스를 양방향(순방향과 역방향)으로 처리하여 과거와 미래의 정보를 모두 활용

→ 기울기 소실 해결 노력

Deep RNN



BiRNN



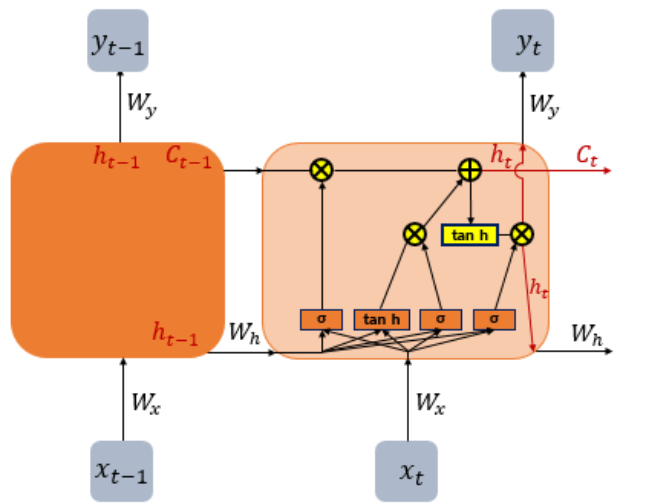
LSTM & GRU

LSTM (Long Short-Term Memory) : 1997년 Hochreiter & Schmidhuber가 제안. RNN의 장기 의존성 문제를 해결하기 위해 셀 상태(cell state)와 3개의 게이트(gate)를 도입. → 망각 게이트 (Forget Gate), 입력 게이트 (Input Gate), 출력 게이트 (Output Gate)

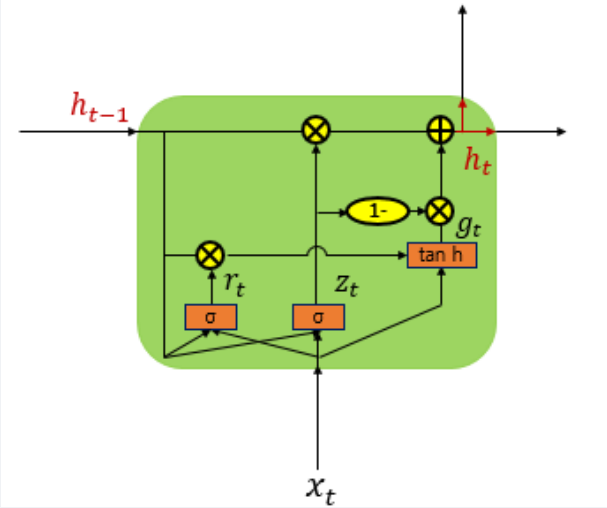
비유: 컨베이어 벨트(셀 상태)에 정보를 실어 보내되, 어떤 정보를 버릴지(망각), 새로 올릴지(입력), 꺼낼지(출력)를 문지기(게이트)가 결정

GRU (Gated Recurrent Unit): LSTM의 간소화.

LSTM



GRU



RNN 활용 패턴 — 입출력 구조별 분류

1 One-to-Many

입력 1개 → 출력 여러 개

예: 이미지 캡셔닝
하나의 이미지를 입력하면 문장을 단어 단위로 생성

"A dog is running on the beach"

2 Many-to-One

입력 여러 개 → 출력 1개

예: 감정 분석, 텍스트 분류
문장 전체를 읽고 긍정/부정 하나를 출력

"이 영화 정말 좋았어" → 긍정

3 Many-to-Many (동일 길이)

입력 N개 → 출력 N개

예: 품사 태깅, 개체명 인식
각 단어에 대해 태그 하나씩 출력

"나는/대명사 학교에/명사 간다/동사"

4 Many-to-Many (Seq2Seq)

입력 N개 → 출력 M개 (길이 다를 수 있음)

예: 기계 번역, 요약
인코더가 입력 시퀀스를 압축하고 디코더가 출력 시퀀스 생성

"나는 학생입니다" → "I am a student"

Seq2Seq — 인코더-디코더 구조

Seq2Seq (Sequence-to-Sequence) 모델

인코더 (Encoder)

입력 시퀀스를 하나의 고정 길이 벡터로 압축

입력: "나는 학생입니다" (단어 단위로 순차 입력)
출력: Context Vector (문맥 벡터)
→ 입력 문장의 정보를 압축한 벡터

→

디코더 (Decoder)

Context Vector를 받아 출력 시퀀스를 생성

입력: Context Vector + <sos> 토큰
출력: "I am a student <eos>"
→ 한 단어씩 자기회귀적(autoregressive)으로 생성

Seq2Seq의 한계

입력 문장이 아무리 길어도 하나의 고정 길이 Context Vector에 압축해야 함
→ 정보 손실 불가피 (병목 현상)
→ 긴 문장에서 성능 저하

해결: Attention 메커니즘

디코더가 출력할 때마다 인코더의 모든 은닉 상태를 참조하여 관련 부분에 "주목"
→ Context Vector 하나에 의존하지 않음
→ 이것이 발전하여 Transformer가 됨

RNN 계열 모델 종합 비교

모델	핵심 특징	장점	한계	활용
기본 RNN	이전 은닉 상태를 다음 입력에 전달	가변 길이 시퀀스 파라미터 공유	장기 의존성 문제 기울기 소실	짧은 텍스트 분류 간단한 시계열
Deep RNN	RNN을 수직으로 여러 층 쌓음	더 복잡한 패턴 높은 추상화	계산 비용 증가 기울기 문제 심화	음성 인식 복잡한 NLP
BiRNN	순방향+역방향 양방향 처리	과거+미래 문맥 모두 활용	전체 시퀀스 필요 실시간 불가	품사 태깅 개체명 인식
LSTM	셀 상태 + 게이트 3개 (망각/입력/출력)	장기 의존성 해결 기울기 소실 완화	파라미터 많음 학습 느림	기계 번역 긴 시계열 예측
GRU	게이트 2개 (업데이트/리셋)	LSTM보다 가벼움 빠른 학습	매우 긴 시퀀스에서 LSTM보다 약할 수 있음	감정 분석 짧은 텍스트

발전 흐름: RNN → Deep RNN → BiRNN → LSTM → GRU → Attention → Transformer

왜 CNN이 필요한가? — 이미지 데이터의 특수성

1 이미지 데이터의 구조

3차원 텐서: 높이 x 너비 x 채널

흑백: 28x28x1 (채널 1개)

컬러(RGB): 28x28x3 (채널 3개)

각 픽셀은 0~255 사이의 정수값→ 이 숫자들의 패턴이 이미지

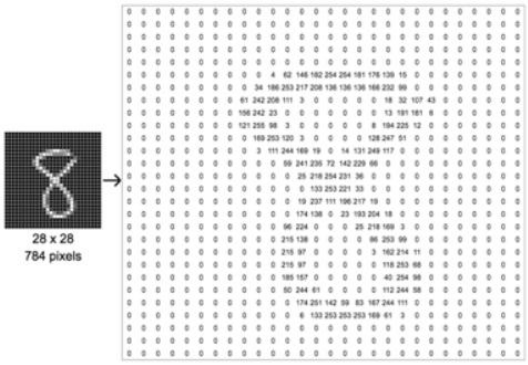
2 일반 신경망으로는 부족?

28x28 이미지 = 784개 입력 뉴런 필요

224x224x3 이미지 = 150,528개!

문제점:

- 파라미터 수 폭발 → 과적합
- 공간적 관계(위치 정보) 무시
- 옆 픽셀, 위아래 픽셀의 관계 소실



3 CNN의 해결 방법

작은 필터(커널)로 이미지를 훑으며 지역적 특징을 추출

파라미터 공유: 같은 필터를 이미지 전체에 적용 → 파라미터 대폭 감소

공간적 구조 보존: 위치 관계 유지

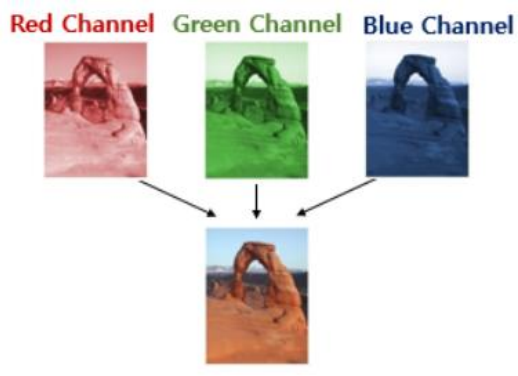
4 생물학적 영감

Hubel & Wiesel(1959) 고양이 실험

시각 피질의 뉴런이 시야의 일부 영역(수용영역)에만 반응

저수준 특징(에지) → 고수준 특징(물체)

이 구조를 모방 → CNN 탄생



합성곱 연산 (Convolution) — 겹쳐서 곱하고 더하기

합성곱 연산이란?

입력 이미지 위에 작은 필터(커널)를 슬라이딩하며 겹치는 부분끼리 곱하고 모두 더하는 연산

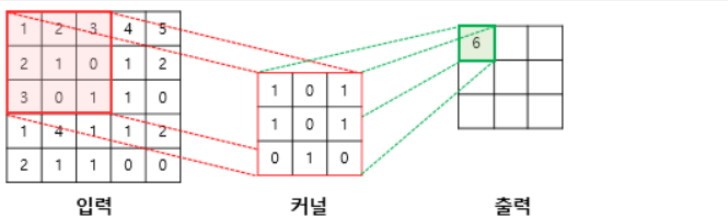
핵심 용어:

커널(Kernel) = 필터(Filter)
- 일반적으로 3x3 또는 5x5 크기
- 학습을 통해 자동으로 최적화됨

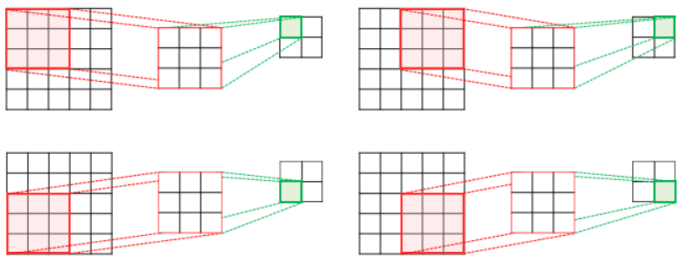
특성 맵(Feature Map)
- 커널을 통과해 출력된 결과 데이터
- 각 커널이 다른 특징을 감지

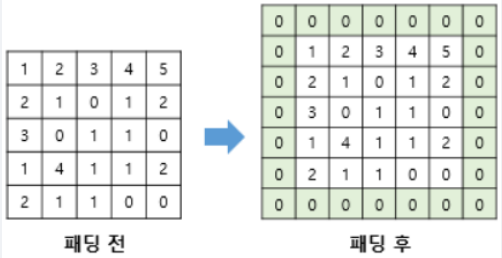
스트라이드(Stride)
- 필터를 이동하는 간격 (보통 1 또는 2)

패딩(Padding)
- 입력 데이터 주변을 0으로 채우기
- 출력 크기를 입력과 동일하게 유지 가능



(1×1) + (2×0) + (3×1) + (2×1) + (1×0) + (0×1) + (3×0) + (0×1) + (1×0) = 6





필터가 감지하는 것

초기 층의 필터: 수평선, 수직선, 대각선 등 에지(edge) 감지
중간 층의 필터: 질감, 패턴, 곡선 등
깊은 층의 필터: 눈, 코, 바퀴 등 복잡한 형태
→ 층이 깊어질수록 더 고수준의 특징 추출

컬러(RGB) 이미지의 경우

필터도 3채널 (R, G, B) 각 채널별 합성곱 결과를 더해서 하나의 특성 맵 생성

CNN 전체 구조

CNN 전체 구조 흐름

1. **입력 이미지** — 224x224x3
→ 2. **합성곱(Conv) + ReLU** — 특징 추출
→ 3. **풀링(Pool)** — 크기 축소
→ 4. **Conv+Pool 반복** — 깊은 특징
→ 5. **Flatten** — 1차원 변환
→ 6. **FC층 + Softmax** — 분류 결과

풀링 (Pooling)

특성 맵의 크기를 줄이는 연산

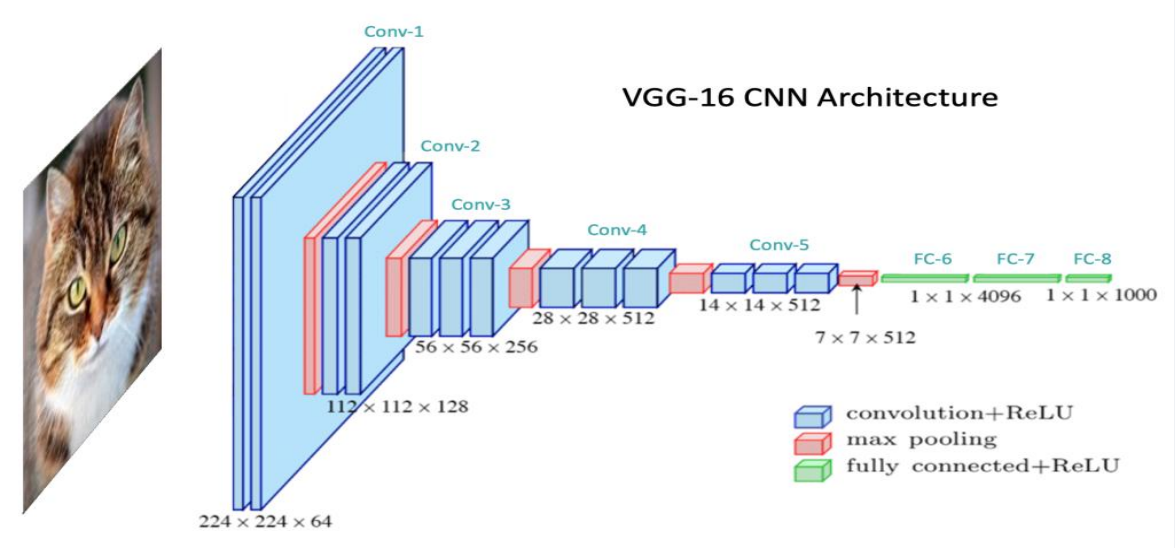
중요한 특징은 유지하면서 불필요한 세부 제거

최대 풀링(Max Pooling). 평균 풀링(Average Pooling)

전결합층 (Fully Connected)

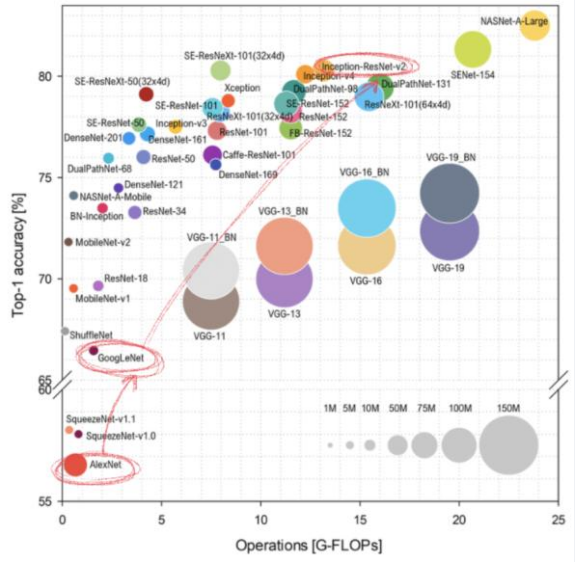
합성곱+풀링으로 추출한 특성 맵을 1차원 벡터로 펼침(Flatten)

예: "고양이 0.85, 개 0.10, 새 0.05"



CNN 주요 모델의 발전

모델	연도	핵심 특징	층 수	의의
LeNet-5	1998	최초의 성공적 CNN. 손글씨 숫자 인식	7	CNN 구조의 원형
AlexNet	2012	ReLU, Dropout, GPU 활용. ImageNet 우승	8	딥러닝 대중화의 기폭제
VGGNet	2014	3x3 작은 필터를 깊게 쌓음	16/19	깊은 네트워크의 효과 입증
GoogLeNet	2014	Inception 모듈 (다양한 필터 크기 병렬)	22	효율적 구조 설계
ResNet	2015	잔차 연결(Skip Connection) 도입	50/101/152	초심층 학습 가능 (기울기 소실 해결)
EfficientNet	2019	너비, 깊이, 해상도를 균형 있게 스케일링	다양	최소 자원으로 최대 성능
ViT	2020	이미지를 패치로 분할 후 Transformer 적용	-	CNN 없이 이미지 분류 가능



흐름: LeNet(CNN 탄생) → AlexNet(GPU+딥러닝) → VGG(깊게) → ResNet(잔차 연결) → ViT(Transformer로 이미지 처리)

CNN 활용 분야

1 이미지 분류 (Classification)

입력 이미지가 어떤 범주에 속하는지 판별

예: 고양이/개 분류, 의료 영상 진단

"이 흉부 X-ray에 폐렴이 있는가?"

모델: VGG, ResNet, EfficientNet

2 객체 탐지 (Object Detection)

이미지 내 물체의 위치(Bounding Box)와 종류를 동시에 판별

예: 자율주행 차량의 보행자 감지

"이 사진에서 사람 3명, 차 2대"

모델: YOLO, Faster R-CNN, SSD

3 이미지 세분화 (Segmentation)

이미지 픽셀이 어떤 클래스에 속하는지 분류 (픽셀 단위 분류)

예: 의료 영상에서 종양 영역 분할

자율주행에서 도로/보행자/차량 구분

모델: U-Net, DeepLab, Mask R-CNN

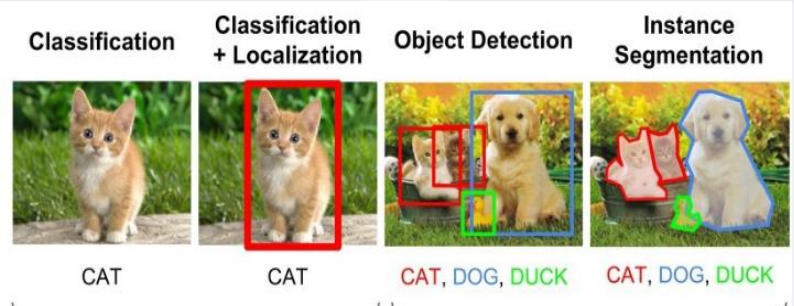
4 미디어 연구 활용

뉴스 영상 프레이밍 분석

정치인 표정/제스처 자동 코딩,

소셜 미디어 이미지 분석(Image Meme 포함)

방송 뉴스 키프레임 추출 후 유형 분류 (앵커/기자/현장 등)



YOLO (You Only Look Once) — 실시간 객체 탐지

YOLO란?

"You Only Look Once" — 이미지를 한 번만 보고 객체의 위치와 종류를 동시에 예측

기존 방식 (R-CNN 계열):
1단계: 후보 영역(Region) 수천 개 생성
2단계: 각 영역을 분류
→ 느림 (2단계 처리)

YOLO 방식:
이미지를 NxN 그리드로 분할
각 그리드 셀이 동시에 객체 예측
→ 한 번의 신경망 통과로 완료
→ 실시간 처리 가능 (30fps 이상)

Ultralytics 라이브러리로 쉽게 사용 가능

YOLO 활용 사례

자율주행: 보행자/차량 실시간 감지
CCTV: 이상 행동 탐지
미디어 연구:
방송 화면에서 인물/로고 감지
시위 장면 군중 분석
의료: X-ray에서 병변 탐지

Python 사용 예시

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")
results = model("image.jpg")
results[0].show()
```

ViT (Vision Transformer) — CNN 없이 이미지 분류

ViT: Transformer를 이미지에 적용 (2020, Google)

1 이미지를 패치로 분할

이미지를 고정 크기의 패치(예: 16x16)로 나눔
224x224 이미지 → 14x14 = 196개 패치
각 패치를 1차원 벡터로 변환(Flatten)

2 패치 임베딩 + 위치 인코딩

각 패치 벡터를 임베딩 차원으로 선형 변환
위치 인코딩(Positional Encoding) 추가
→ 패치의 순서(위치) 정보 보존

3 Transformer 인코더에 입력

패치 시퀀스를 NLP의 단어 시퀀스처럼 처리
Self-Attention으로 패치 간 관계 학습
→ 멀리 떨어진 패치도 직접 관련성 파악 가능

4 [CLS] 토큰으로 분류

BERT처럼 [CLS] 토큰 추가
이 토큰의 최종 표현을 FC층에 넣어 분류
→ CNN 없이도 이미지 분류 가능!

> 의의: "이미지를 단어 시퀀스처럼 처리" → NLP와 컴퓨터 비전의 통합. 대규모 데이터에서 CNN보다 우수한 성능. Transformer가 NLP뿐만 아니라 Vision까지 지배하는 시대를 열었음. (Part 3 Transformer에서 상세 설명)

텍스트 이미지 학습데이터셋 찾기

사이트	URL	특징
Hugging Face Datasets	huggingface.co/datasets	NLP+CV 통합, load_dataset() API
Kaggle	kaggle.com/datasets	커뮤니티 데이터셋, Competition 연계
Papers With Code	paperswithcode.com/datasets	논문-데이터셋-리더보드 연결
TensorFlow Datasets	tensorflow.org/datasets	tfds.load() API, 표준 포맷
Ultralytics Docs	docs.ultralytics.com/datasets	YOLO 학습용 COCO/VOC/Open Images, yaml 설정 제공
Roboflow Universe	universe.roboflow.com	10만+ 커스텀 데이터셋, YOLO/COCO 포맷 자동 변환
Google Dataset Search	datasetsearch.research.google.com	메타 검색엔진 (전 분야 통합 검색)
FiftyOne (Voxel51)	docs.voxel51.com	COCO/Open Images 부분 다운로드·시각화, 평가 도구
ICDAR RRC	rrc.cvc.uab.es	텍스트 검출·인식 벤치마크 (ICDAR 대회 데이터)
▼ **한국어 특화**		
AI Hub (한국어)	aihub.or.kr	한국어 NLP/CV 대규모 데이터 (감성분석, NER 등)
KLUE	klue-benchmark.com	한국어 NLU 벤치마크 8개 태스크 (NER, RE, NLI 등)
모두의 말뭉치	corpus.korean.go.kr	국립국어원 말뭉치 (형태소, 구문, 개체명 등)

RNN vs CNN — 핵심 비교

	RNN 계열	CNN 계열
대상 데이터	시퀀스 (텍스트, 음성, 시계열)	이미지 (2D/3D 공간 데이터)
핵심 연산	순환(Recurrent) — 시간축 따라 전달	합성곱(Convolution) — 필터로 특징 추출
기억/문맥	은닉 상태로 이전 정보 기억	수용 영역(Receptive Field) 내 공간 패턴
병렬 처리	어려움 (순차 처리 필수)	가능 (필터 동시 적용)
대표 모델	LSTM, GRU, BiRNN	ResNet, VGG, YOLO, EfficientNet
현재 추세	Transformer로 대체되는 중	ViT(Transformer)로 대체되는 중

> 공통점: 둘 다 Transformer에 의해 대체되고 있음. RNN → Transformer(NLP), CNN → ViT(Vision)